

APIOBike

APIOBike เป็นบริการให้ยืมจักรยานที่เพิ่งเปิดตัวใน APIO City โดยมีการติดตั้งสถานีกระจายไปในเมือง เพื่อให้ผู้ใช้สามารถยืมและคืนจักรยานได้ที่สถานีใดก็ได้ตามสะดวก เนื่องด้วยความสะดวกและราคาที่ต่ำ APIOBike จึงกลายเป็นรูปแบบการเดินทางที่สำคัญที่สุดในเมืองนี้ อย่างไรก็ตาม APIOBike ได้พบกับปัญหาที่เจอกันทั่วไปในธุรกิจประเภทนี้ นั่นคือ ความไม่สมดุลระหว่างการยืมและการคืนที่สถานีต่าง ๆ ปัญหานี้ทำให้บางสถานีมีรถจักรยานจำนวนน้อยมาก ทำให้ผู้คนที่อยู่แถวสถานีนั้นไม่สามารถยืมได้ และในขณะเดียวกัน ก็มีบางสถานีที่มีจักรยานเหลือมากเกินไปจนความต้องการของคนที่อยู่ในเขตสถานีนั้น ๆ

เพื่อแก้ปัญหา APIOBike วางแผนที่จะส่งรถบรรทุกให้ไปกระจายจักรยานตามสถานีต่าง ๆ เพื่อรับประกันว่าแต่ละสถานีจะมีจักรยานตามจำนวนที่เหมาะสม APIO City มีสถานีจำนวน N สถานี หมายเลข $0, 1, \dots, N - 1$ จากการสังเกต APIOBike พบว่าในทุก ๆ เวลา ค่า จำนวนรถจักรยานที่สถานี i จะมีค่าเท่ากับค่าคงที่ $A[i]$ เสมอ และไม่มีใครยืมหรือคืนจักรยานในเวลากลางคืน บริษัทตั้งเป้าว่าจะต้องมีจักรยานจำนวน $B[i]$ คนที่สถานี i ในเวลาเช้านี้

ระหว่างสถานีจำนวน N สถานีเหล่านี้ จะมีถนนเฉพาะสำหรับรถบรรทุกกระจายจักรยานโดยเฉพาะจำนวน $N - 1$ เส้น ถนนแต่ละเส้นเชื่อมสถานีสองสถานีที่แตกต่างกัน และรถบรรทุกจะสามารถวิ่งไปตามถนนเหล่านี้ได้เท่านั้น ถนนแต่ละเส้นยาวหนึ่งหน่วย โครงข่ายถนนนี้เชื่อมต่อกัน และรับประกันว่ารถบรรทุกสามารถเดินทางได้ระหว่างคู่ของสถานีใด ๆ ทุก ๆ คือ APIOBike จะต้องส่งรถบรรทุกกระจายจักรยานเพื่อรับประกันว่าแต่ละสถานี i จะต้องมียาน $B[i]$ คัน รถสามารถเริ่มที่สถานีใดก็ได้ และจะจบการเดินทางที่สถานีใดก็ได้

เมื่อแรกเริ่มกระบวนการปรับสมดุล รถบรรทุกจะว่างเปล่า เมื่อใดก็ตามที่รถอยู่ที่สถานี คนขับรถจะสามารถนำรถจักรยานที่คืนก็ได้จากสถานีขึ้นมาบนรถ หรือนำรถจักรยานที่คืนก็ได้จากบนรถบรรทุกไปไว้ที่สถานี ทั้งรถบรรทุกและสถานีมีที่เก็บจักรยานไม่จำกัดจำนวน อย่างไรก็ตาม จำนวนรถจักรยานที่สถานีใด ๆ จะต้องไปลดลงต่ำกว่าศูนย์ บริษัทต้องการหาว่าระยะทางที่รถบรรทุกจะต้องใช้น้อยที่สุดเพื่อจะปรับจำนวนจักรยานให้เป็นไปตามเงื่อนไขเป็นเท่าใด รวมทั้งกระบวนการปรับความสมดุลนั้นด้วย

Implementation Details

คุณจะต้องเขียนฟังก์ชันต่อไปนี้

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A : อาร์เรย์ความยาว N , โดยที่ $A[i]$ เป็นจำนวนจักรยานที่สถานี i ในแต่ละคำ.
- B : อาร์เรย์ความยาว N , โดยที่ $B[i]$ เป็นจำนวนจักรยานเป้าหมายที่สถานี i ในทุกเช้า.
- U, V : อาร์เรย์ความยาว $N - 1$ ที่แทนเครือข่ายถนน สำหรับแต่ละ $0 \leq i < N - 1$, ถนนเส้นที่ i เชื่อมสถานี $U[i]$ กับสถานี $V[i]$
- ฟังก์ชันนี้จะถูกเรียก **ไม่เกิน 150 000 ครั้ง** ในแต่ละกรณีทดสอบ.

ฟังก์ชันจะต้องคืนคู่ลำดับของอาร์เรย์ (X, Y) ที่มีความยาวเท่ากันที่เท่ากับ $k + 1$ ที่แทนกลยุทธ์การปรับสมดุลจักรยาน:

- X : ลำดับของสถานีที่จะต้องไปแวะตามลำดับก่อนหลัง.
- Y : จำนวนจักรยานสุทธิที่ต้องส่งในแต่ละสถานีที่แวะตามลำดับ กล่าวคือ สำหรับ $0 \leq j \leq k$:
 - ถ้า $Y[j] \geq 0$, คนขับจะวางจักรยานจำนวน $Y[j]$ คันที่สถานี $X[j]$ จากถนนรถทุก ทำให้จำนวนจักรยานที่สถานีเพิ่มขึ้นเท่ากับ $Y[j]$
 - ถ้า $Y[j] < 0$, คนขับจะเก็บจักรยานจำนวน $-Y[j]$ คัน จากสถานี $X[j]$ ไปยังถนนรถทุก ทำให้จำนวนจักรยานที่สถานีลดลงเท่ากับ $-Y[j]$.

กลยุทธ์การปรับสมดุลจักรยาน (X, Y) ที่มีระยะการเดินทาง k จะ **ถูกต้อง** ถ้ากลยุทธ์นั้น สอดคล้องกับเงื่อนไขต่อไปนี้

- สำหรับแต่ละ $0 \leq j \leq k, 0 \leq X[j] < N$.
- สำหรับแต่ละ $0 \leq j < k$, สถานี $X[j]$ และสถานี $X[j + 1]$ เชื่อมต่อกันด้วยถนน
- สำหรับแต่ละ $0 \leq j \leq k, \sum_{t=0}^j Y[t] \leq 0$.
- สำหรับแต่ละ $0 \leq i < N$ และแต่ละขั้นตอน $0 \leq j \leq k$, ให้ $S_{i,j}$ คือผลรวมของแต่ละ $Y[t]$ สำหรับทุก ๆ ขั้นตอน t ที่ $0 \leq t \leq j$ และ $X[t] = i$.
 - จำนวนจักรยานที่สถานี i จะไม่ลดลงไปน้อยกว่าศูนย์: $A[i] + S_{i,j} \geq 0$.
 - เมื่อกลยุทธ์การปรับสมดุลสิ้นสุดลง, แต่ละสถานี i จะต้องมียานจำนวนเท่ากับ $B[i]$ คันพอดี: $A[i] + S_{i,k} = B[i]$.

Constraints

- $2 \leq N \leq 300\,000$
- ผลรวมของ N ในทุก ๆ การเรียกใช้ `find_rebalancing_strategy` จะไม่เกิน 300 000 ในแต่ละกรณีทดสอบ.
- $0 \leq U[i], V[i] < N$ และ $U[i] \neq V[i]$ สำหรับแต่ละ i ที่ $0 \leq i < N$.
- $0 \leq A[i], B[i] \leq 10^9$ สำหรับแต่ละ i ที่ $0 \leq i < N$.
- สามารถเดินทางระหว่างคู่สถานีใด ๆ ได้เสมอ.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- จะมีบาง i ที่ $0 \leq i < N$ และ $A[i] \neq B[i]$.

Subtasks and Scoring

ในที่นี้ ให้ T แทนจำนวนครั้งของการเรียกฟังก์ชัน `find_rebalancing_strategy`, และ $\sum N$ แทนผลรวมของ N ทั้งหมดในการเรียก `find_rebalancing_strategy`.

Subtask	คะแนน	เงื่อนไขเพิ่มเติม
1	4	สถานีและถนนสอดคล้องกับคุณสมบัติ P (ดูด้านล่าง). มีสถานี i เดียวเท่านั้นที่ $0 \leq i < N$ และ $A[i] > 0$.
2	11	$T \leq 10$. $N \leq 7$. สถานีและถนนสอดคล้องกับคุณสมบัติ P.
3	16	สถานีและถนนสอดคล้องกับคุณสมบัติ P.
4	9	มีสถานี i เดียวเท่านั้นที่ $0 \leq i < N$ และ $A[i] > 0$.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	ไม่มีเงื่อนไขเพิ่มเติมอื่น ๆ.

คุณสมบัติ P: สำหรับแต่ละ $0 \leq i < N - 1$, $U[i] = i$ และ $V[i] = i + 1$. สำหรับ $0 \leq i < N$, $A[i] \neq B[i]$.

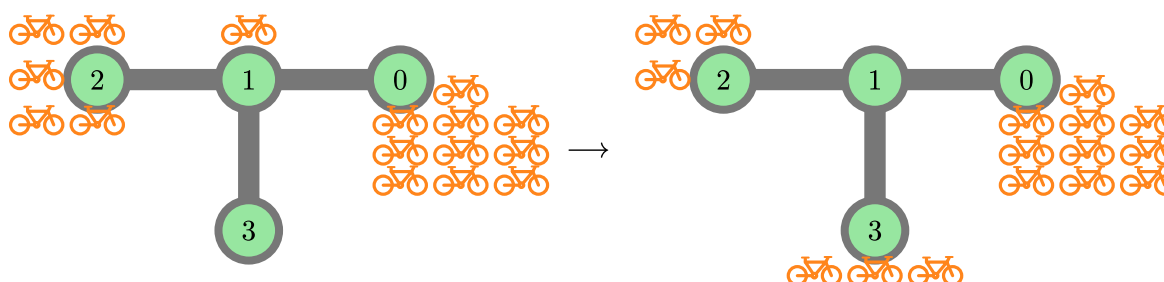
ถ้าอาร์เรย์ X และ Y ที่คืนกลับมา มีความยาวเท่ากับ $k^* + 1$ พอดี โดยที่ k^* คือความยาวที่น้อยที่สุดที่เป็นไปได้ในการเดินทาง แต่ (X, Y) ไม่ใช่กลยุทธ์ถูกต้อง คุณจะได้คะแนนเท่ากับ 50% ของคะแนนเต็ม.

Example

Example 1

พิจารณาการเรียกต่อไปนี้:

```
find_rebalancing_strategy(4,
                        [10, 1, 5, 0],
                        [10, 0, 3, 3],
                        [0, 1, 1],
                        [1, 2, 3])
```



มีสถานีเท่ากับ $N = 4$ สถานีใน APIO City ในตอนแรก สถานีมีจักรยานเท่ากับ $A = [10, 1, 5, 0]$ และเป้าหมายคือต้องการมีจักรยานเท่ากับ $B = [10, 0, 3, 3]$ ในทุก ๆ สถานี

สมมติว่า รถบรรทุกสำหรับปรับสมดุลย์เริ่มที่สถานี 2 และดำเนินการตามขั้นตอนต่อไปนี้:

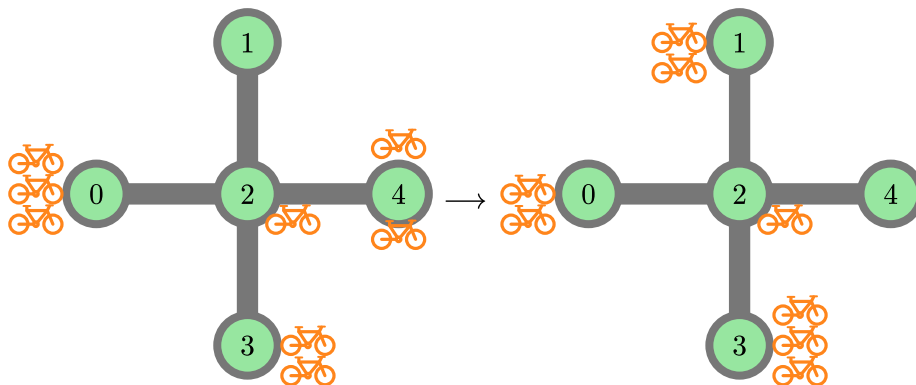
- ที่สถานี 2, นำจักรยานจำนวน 2 คัน ขึ้นไปบนรถบรรทุก. สถานี 2 จะเหลือจักรยาน 3 คัน และรถบรรทุกมีจักรยาน 2 คัน.
- เดินทางไปสถานี 1 และนำจักรยานจำนวน 1 คัน ขึ้นไปบนรถบรรทุก. สถานี 1 จะเหลือจักรยาน 0 คัน และรถบรรทุกมีจักรยาน 3 คัน.
- เดินทางไปสถานี 3, และนำจักรยานจำนวน 3 คัน ลงจากรถบรรทุก. สถานี 3 จะมีจักรยาน 3 คัน และรถบรรทุกมีจักรยาน 0 คัน.

ระยะการเดินทางทั้งสิ้นจะเท่ากับ 2 อาร์เรย์ที่จะต้องคืนค่าสำหรับกลยุทธีนี้คือ $X = [2, 1, 3]$ และ $Y = [-2, -1, 3]$ สามารถพิสูจน์ได้ว่ากลยุทธีนี้เป็นกลยุทธีที่ใช้ระยะการเดินทางที่สั้นที่สุด ดังนั้นฟังก์ชันควรต้องคืนค่า $([2, 1, 3], [-2, -1, 3])$.

Example 2

พิจารณาการเรียกต่อไปนี้:

```
find_rebalancing_strategy(5,  
                          [3, 0, 1, 2, 2],  
                          [2, 2, 1, 3, 0],  
                          [2, 2, 2, 2],  
                          [0, 4, 3, 1])
```



มีสถานีจำนวน $N = 5$ สถานีใน APIO City. เมื่อเริ่มต้น สถานีมีรถจักรยานจำนวน $A = [3, 0, 1, 2, 2]$ และมีเป้าหมายที่ต้องการมีจักรยานจำนวน $B = [2, 2, 1, 3, 0]$ ที่แต่ละสถานี

สมมติว่ารถบรรทุกปรับสมดุลย์เริ่มที่สถานี 0 และดำเนินการตามขั้นตอนด้านล่าง:

- ที่สถานี 0, นำจักรยานจำนวน 1 คัน ขึ้นไปบนรถบรรทุก.
- ไปยังสถานี 2 และนำจักรยานจำนวน 1 คัน ขึ้นไปบนรถบรรทุก.
- ไปยังสถานี 1 และนำจักรยานจำนวน 2 คัน ลงจากรถบรรทุก.
- ไปยังสถานี 2.

- ไปยังสถานี 4 และนำจักรยานจำนวน 2 คัน ขึ้นไปบนรถบรรทุก.
- ไปยังสถานี 2 และนำจักรยานจำนวน 1 คัน ลงจากรถบรรทุก.
- ไปยังสถานี 3 และนำจักรยานจำนวน 1 คัน ลงจากรถบรรทุก.

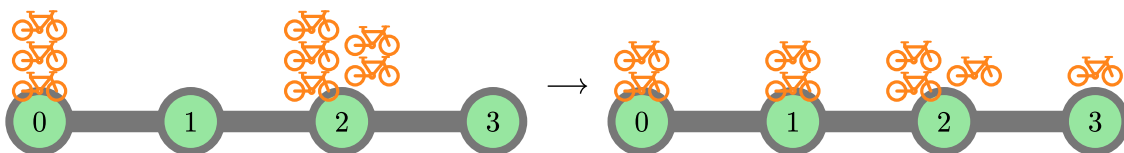
ระยะการเดินทางรวมคือ 6 อาร์เรย์ที่จะต้องคืนค่าสำหรับกลยุทธ์นี้คือ $X = [0, 2, 1, 2, 4, 2, 3]$ และ $Y = [-1, -1, 2, 0, -2, 1, 1]$. สามารถพิสูจน์ได้ว่ากลยุทธ์นี้เป็นกลยุทธ์ที่ใช้ระยะการเดินทางที่สั้นที่สุด ดังนั้นฟังก์ชันควรต้องคืนค่า $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$.

กลยุทธ์ที่ถูกต้องอื่น ๆ ที่มีระยะการเดินทางเท่ากับ 6 เช่น $X = [0, 2, 3, 2, 4, 2, 1]$ และ $Y = [-1, 0, 1, 0, -2, 0, 2]$ จะนับว่าเป็นคำตอบที่ถูกต้องเช่นเดียวกัน การคืนอาร์เรย์ X, Y ที่มีความยาว $7 = 6 + 1$ แต่ไม่ใช่กลยุทธ์ที่ถูกต้อง เช่น $X = Y = [0, 0, 0, 0, 0, 0, 0]$ จะทำให้ได้คะแนน 50% ของคะแนนเต็ม

Example 3

พิจารณาการเรียกต่อไปนี้:

```
find_rebalancing_strategy(4,
                        [3, 0, 5, 0],
                        [2, 2, 3, 1],
                        [0, 1, 2],
                        [1, 2, 3])
```



คำตอบที่ดีที่สุดคือ $X = [2, 1, 0, 1, 2, 3]$ และ $Y = [-1, 1, -1, 1, -1, 1]$ ฟังก์ชันควรคืนค่า $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$. กลยุทธ์อื่น ๆ ที่ถูกต้อง เช่น $X = [2, 1, 0, 1, 2, 3]$ และ $Y = [-2, 2, -1, 0, 0, 1]$, จะถือว่าเป็นคำตอบที่ถูกต้องเช่นเดียวกัน

ตัวอย่างนี้ สอดคล้องกับเงื่อนไขในปัญหาย่อย 2 และ 3

Sample Grader

บรรทัดแรกของข้อมูลนำเข้าจะต้องระบุจำนวนเต็ม T แทนจำนวนสถานการณ์ จากนั้นรายละเอียดของสถานการณ์จำนวน T สถานการณ์จะตามมา ในรูปแบบต่อไปนี้

รูปแบบข้อมูลนำเข้า:

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

รูปแบบข้อมูลส่งออก:

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```